

Vectors, Vector Spaces & Matrices for AI/ML

A rigorous foundation for undergraduate Mathematics in Artificial Intelligence and Machine Learning

Section 1

Vectors – representation, operations, and ML applications

Section 2

Vector Spaces – axioms, subspaces, basis, and dimension

Section 3

Matrices – structure, operations, transformations, and ML use

Integration

Complete worked example and final concept map

Section 1

Vectors

Magnitude, direction, operations, and the building blocks of every ML model

01

What Is a Vector?

Definition, components, geometric meaning

03

Dot Product

Algebraic and geometric interpretations

02

Vector Operations

Addition, scalar multiplication, norm, distance

04

Vectors in ML

Linear models, embeddings, cosine similarity

What Is a Vector?

Magnitude and Direction

A vector encodes both **how much** and **which way** — it is not merely a quantity but a directed quantity in space.

Ordered List of Numbers

Each entry is a **component** along a coordinate axis. The ordering is essential — swapping entries produces a different vector.

Scalar vs. Vector

A **scalar** is a single real number (e.g., temperature = 37). A **vector** is an ordered tuple of numbers (e.g., velocity = (3, 4)).

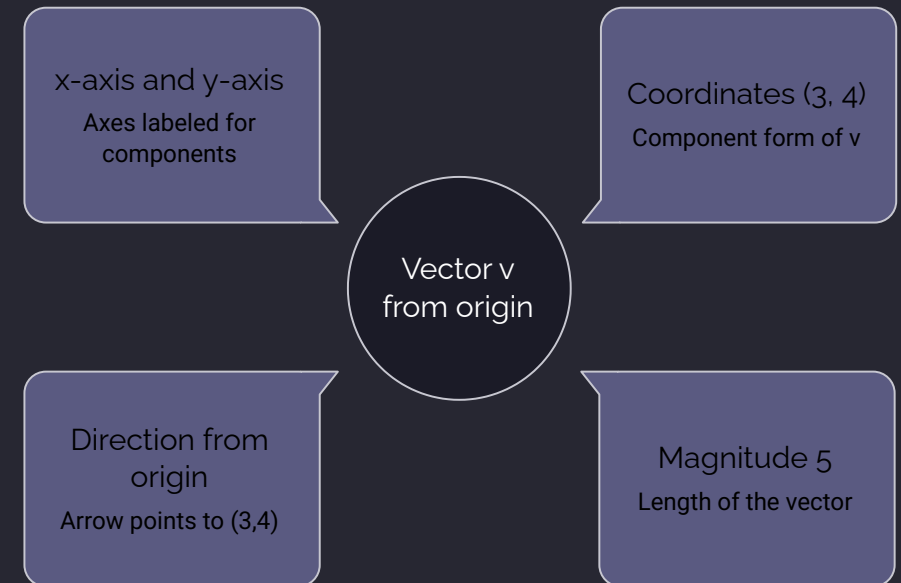
Physical Examples

Movement, velocity, force, and position are all naturally vector quantities — they require both magnitude and direction to be fully specified.

Example Vector

The canonical example: $\mathbf{v} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$

Drawn as an arrow from the origin (0, 0) to the point (3, 4) on a coordinate plane.



The horizontal component is 3 (along the x-axis) and the vertical component is 4 (along the y-axis).

Components and Geometric Meaning

General Form

A vector in $\mathbb{R}^n$: $\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}$ — each v_i is the component along axis i , measuring displacement in that coordinate direction.

2D Vectors

Components measure displacement along x and y axes in the plane.

3D Vectors

Three components for displacement along x, y, and z in ordinary space.

Point vs. Arrow

A **point** marks a fixed location in space. A **vector** is a directed displacement — an arrow that can be placed anywhere, defined only by its magnitude and direction, not its starting position.

- In ML, a **feature vector** $\mathbf{x} \in \mathbb{R}^n$ has one component per feature — e.g., age, income, credit score. The vector $(3, 4)$ drawn from the origin illustrates both the point and the arrow interpretation simultaneously.

Higher Dimensions

In machine learning, n can be thousands or millions. A document embedding might live in $\mathbb{R}^{768}$; an image patch in $\mathbb{R}^{512}$. The geometric intuition from 2D and 3D extends formally to all dimensions.

Vector Operations

Vector Addition

$$\mathbf{u} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \Rightarrow \mathbf{u} + \mathbf{v} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

Add corresponding components element-wise. Geometrically, place vectors **tip-to-tail**; the diagonal of the resulting parallelogram is the sum vector.

Scalar Multiplication

$$2 \cdot \begin{bmatrix} 3 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 6 \\ 3 \end{bmatrix}$$

Multiply every component by the scalar. The direction is preserved; only the magnitude changes.

Effect of the Scalar

Positive scalar ($c > 0$)

Same direction, magnitude scales proportionally. The vector stretches or shrinks along its original direction.

Negative scalar ($c < 0$)

Direction reverses. The vector points in the opposite direction with magnitude $|c| \|\mathbf{v}\|$.

Zero scalar ($c = 0$)

Collapses the vector to the **zero vector** $\mathbf{0}$ — no magnitude, no direction.

Norm and Distance

Euclidean Norm (L2)

The **norm** of a vector measures its length in space:

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

Worked example:

$$\mathbf{v} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \Rightarrow \|\mathbf{v}\| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

Distance Between Points

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|$$

Subtract the vectors, then take the norm of the difference. This is the straight-line (Euclidean) distance between two points in $\mathbb{R}^n$.

ML Applications of Norms

k-NN Classification

Assigns a label based on the k nearest neighbors by Euclidean distance

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|.$$

Clustering (k-Means)

Assigns each point to the cluster whose centroid is closest – again measured by Euclidean distance.

L2 Regularization (Ridge)

Penalizes $\|\mathbf{w}\|^2$ in the loss function to prevent overfitting by keeping weights small.

L1 Regularization (Lasso)

Penalizes $\|\mathbf{w}\|_1 = \sum_i |w_i|$, promoting sparsity – many weights become exactly zero.

Dot Product

Algebraic Definition

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^n u_i v_i$$

Worked example:

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 4 \end{bmatrix} = 1(3) + 2(4) = 3 + 8 = 11$$

Geometric Form

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$$

Where θ is the angle between the two vectors. This connects the algebraic computation to the geometric relationship between directions.

Interpreting the Sign of the Dot Product

Positive ($\mathbf{u} \cdot \mathbf{v} > 0$)

Vectors point in **similar directions**. The angle $\theta < 90^\circ$, so $\cos \theta > 0$. High alignment.

Zero ($\mathbf{u} \cdot \mathbf{v} = 0$)

Vectors are **perpendicular (orthogonal)**. The angle $\theta = 90^\circ$, so $\cos \theta = 0$. No shared direction.

Negative ($\mathbf{u} \cdot \mathbf{v} < 0$)

Vectors point in **opposing tendencies**. The angle $\theta > 90^\circ$, so $\cos \theta < 0$. Anti-aligned.

📌 The dot product is the fundamental operation underlying nearly all of linear algebra and ML — from projections to neural network activations.

Vectors in Machine Learning

Core Linear Model

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

The prediction is a **dot product** of the weight vector $\mathbf{w}$ and the feature vector $\mathbf{x}$, plus a scalar bias b . This single equation underlies linear regression, logistic regression, and the first layer of any neural network.

Linear Regression

Minimizes the squared residual norm: $\|\mathbf{y} - X\mathbf{w}\|^2$. Dot products appear in every gradient computation and in the closed-form solution $\mathbf{w}^* = (X^T X)^{-1} X^T \mathbf{y}$.

Logistic Regression

Applies the sigmoid function to the linear combination: $\hat{p} = \sigma(\mathbf{w}^T \mathbf{x} + b)$, mapping the dot product to a probability in $(0, 1)$ for binary classification.

Neural Network Neurons

Each neuron computes a **weighted sum** (dot product) of its inputs, then applies a non-linear activation function. A layer of neurons is a matrix-vector multiplication followed by an element-wise activation.

Cosine Similarity & Embeddings

$$\text{cosine similarity} = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

Normalizes the dot product to $[-1, 1]$, measuring directional alignment regardless of magnitude. Used extensively in NLP (word2vec, BERT) and recommendation systems to compare embedding vectors.

Cross Product (3D Only)

Produces a vector **perpendicular** to both inputs; its magnitude equals the area of the parallelogram spanned by the two vectors. Less central in mainstream ML but appears in 3D graphics and robotics.

Section 2

Vector Spaces

The mathematical environment that makes linear algebra — and machine learning — work

01

Definition & Intuition

What a vector space is and why we need one

03

Examples & Non-Examples

Recognizing valid and invalid vector spaces

02

Axioms

The eight properties every vector space must satisfy

04

Subspaces, Basis & Dimension

Structure within spaces and the tools for dimensionality

What Is a Vector Space?

Core Intuition

A **vector space** is a mathematical environment where vectors can be added together and scaled by scalars, and the results always remain inside the same set. No operation can "escape" the space.

"A vector space is a mathematical playground where vector operations always remain valid."

Why We Need It

Without a common, well-defined setting, operations like $\mathbf{u} + \mathbf{v}$ or $c\mathbf{v}$ have no guaranteed meaning. The vector space framework provides the axioms that make these operations reliable and predictable — a prerequisite for any rigorous mathematical or computational work.

Valid Operations

Both **addition** and **scalar multiplication** must satisfy **closure**: applying either operation to elements of the space must produce another element of the same space. This is the most fundamental requirement.

Foundation for Machine Learning

Every core ML structure is a vector space:

Feature Space

$\mathbb{R}^p$ — where each data point lives as a feature vector

Weight Space

$\mathbb{R}^d$ — where model parameters are updated during training

Embedding Space

$\mathbb{R}^k$ — where words, images, or users are represented as dense vectors

This structure guarantees that **gradient descent** and all linear algebra tools work correctly within these spaces.

Definition and Main Axioms

A **vector space** is a set V closed under vector addition and scalar multiplication, satisfying all of the following axioms for all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V$ and scalars $c, d \in \mathbb{R}$:

1

Closure under Addition

$$\mathbf{u} + \mathbf{v} \in V$$

2

Closure under Scalar Mult.

$$c\mathbf{v} \in V$$

1

Zero Vector

There exists $\mathbf{0}$ such that
 $\mathbf{v} + \mathbf{0} = \mathbf{v}$

2

Additive Inverse

For every $\mathbf{v}$, there exists $-\mathbf{v}$
such that $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$

3

Commutativity

$$\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$$

4

Associativity

$$(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$$

3

Scalar Identity

$$1 \cdot \mathbf{v} = \mathbf{v}$$

4

Distributive Properties

$$c(\mathbf{u} + \mathbf{v}) = c\mathbf{u} + c\mathbf{v} \text{ and} \\ (c + d)\mathbf{v} = c\mathbf{v} + d\mathbf{v}$$

☐ All eight axioms must hold simultaneously. Failing even one disqualifies the set from being a vector space.

Examples and Non-Example

Valid Vector Spaces

$$\mathbb{R}^2$$

All 2D vectors – the standard Cartesian plane. The most familiar vector space.

$$\mathbb{R}^3$$

All 3D vectors – ordinary three-dimensional space. Intuition from here extends to all dimensions.

$$\mathbb{R}^n$$

All n -dimensional vectors – the primary setting for ML feature vectors and weight vectors.

Polynomial Spaces

All polynomials of degree $\leq n$ form a vector space under polynomial addition and scalar multiplication.

Fixed-Size Matrices

The set of all $m \times n$ matrices is a vector space – addition and scalar multiplication are defined entry-wise.

Non-Example: The Positive Quadrant

Consider the set:

$$S = \left\{ \begin{bmatrix} x \\ y \end{bmatrix} : x > 0, y > 0 \right\}$$

This is the set of all 2D vectors with strictly positive components – the open first quadrant.

Why It Fails

Take any vector $\mathbf{v} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \in S$. Multiply by -1 :

$$(-1)\mathbf{v} = \begin{bmatrix} -1 \\ -1 \end{bmatrix} \notin S$$

The result has negative components and lies **outside** S . Closure under scalar multiplication fails – therefore S is **not** a vector space.

Subspaces

A **subspace** is a subset $W \subseteq V$ of a vector space that is itself a vector space under the same addition and scalar multiplication operations inherited from V .

Canonical Example: The x-Axis in $\mathbb{R}^2$

$$W = \left\{ \begin{bmatrix} x \\ 0 \end{bmatrix} \in \mathbb{R}^2 : x \in \mathbb{R} \right\}$$

Geometrically, W is the horizontal axis — a line through the origin. Adding two vectors of this form gives another vector with zero second component. Scaling one does the same. The origin $\begin{bmatrix} 0 \\ 0 \end{bmatrix}$ is in W . All three subspace conditions are satisfied.

The Three-Condition Subspace Test

1 Contains the zero vector

$\mathbf{0} \in W$ — the zero vector must belong to the subset.

2 Closed under addition

$$\mathbf{u}, \mathbf{v} \in W \Rightarrow \mathbf{u} + \mathbf{v} \in W$$

3 Closed under scalar multiplication

$$\mathbf{v} \in W, c \in \mathbb{R} \Rightarrow c\mathbf{v} \in W$$

ML Relevance of Subspaces

Column Space

The column space of a weight matrix W is a subspace — it contains all possible outputs of the linear transformation $W\mathbf{x}$. It determines what the model **can** represent.

Null Space

The null space of a linear map contains all inputs that map to $\mathbf{0}$. It reveals what the model **cannot distinguish** — inputs in the null space produce identical outputs.

Row Space

The row space captures the directions in input space that actually influence the output. Orthogonal to the null space.

- Understanding column and null spaces explains the expressive power and limitations of any linear model or neural network layer.

Linear Combinations, Span, and Basis

Key Definitions

Linear Combination

$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \dots + c_n\mathbf{v}_n$ — combining vectors with scalar weights $c_i \in \mathbb{R}$.

Span

The set of **all possible** linear combinations of a collection of vectors — the entire "reach" of those vectors in the space.

Linear Independence

A set of vectors is linearly independent if no vector in the set can be written as a linear combination of the others. No redundancy.

Basis

A **linearly independent** set that **spans** the entire space — the minimal building blocks. Every vector has a unique representation in terms of the basis.

Standard Basis of $\mathbb{R}^2$

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Every 2D vector is a unique linear combination: $\begin{bmatrix} 3 \\ 4 \end{bmatrix} = 3\mathbf{e}_1 + 4\mathbf{e}_2$. The coefficients are exactly the components.

Dimension

The **dimension** of a vector space is the number of vectors in any basis. $\mathbb{R}^n$ has dimension n . This is well-defined — all bases of the same space have the same number of vectors.

ML Relevance: PCA

Principal Component Analysis (PCA) finds a **new basis** for the data — one that captures the most variance with the fewest vectors. Projecting onto the top k principal components reduces dimension from n to k while preserving the most information.

Section 3

Matrices

Rectangular arrays that store data, encode transformations, and carry learned parameters

01

Structure & Types

Definition, notation, and common matrix forms

03

Properties

Non-commutativity, transpose, symmetry

02

Operations

Addition, scalar multiplication, and matrix multiplication

04

ML Applications

Datasets, neural networks, images, recommendations

What Is a Matrix?

Definition

A **matrix** is a rectangular array of numbers arranged in rows and columns. It is the primary data structure of linear algebra and the backbone of all ML computations.

Example: A 2×3 Matrix

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Rows

Horizontal slices – 2 rows here. Indexed top to bottom.

Columns

Vertical slices – 3 columns here. Indexed left to right.

Elements

Entry a_{ij} is in row i , column j .
Example: $a_{12} = 2$.

Order / Size

Written as $m \times n$ (rows $\times$ columns). Always rows first.

Why Matrices?

Matrices compactly represent four fundamentally different things in ML:

Systems of Equations

A system of m linear equations in n unknowns is encoded as $A\mathbf{x} = \mathbf{b}$.

Datasets

An $n \times p$ data matrix stores n samples, each with p features.

Linear Transformations

Multiplying by a matrix rotates, scales, reflects, or projects vectors.

Learned Parameters

Weight matrices in neural networks store all learned parameters for a layer.

Matrices as Datasets

The Data Matrix

$$X = \begin{bmatrix} 20 & 85 & 90 \\ 21 & 78 & 88 \\ 19 & 92 & 95 \end{bmatrix}$$

A 3×3 dataset with 3 students (rows) and 3 attributes (columns).

Student	Age	Quiz	Exam
1	20	85	90
2	21	78	88
3	19	92	95

Interpreting the Structure

Rows = Observations

Each row is one data sample – one student, one transaction, one image. Row i is the complete record for sample i .

Columns = Features

Each column is one measured attribute – age, quiz score, exam score. Column j contains all values for feature j across all samples.

Row as Feature Vector

Each row $\mathbf{x}_i \in \mathbb{R}^p$ is the feature vector for sample i . The entire dataset is the collection of these vectors stacked into a matrix.

ML Convention

$X \in \mathbb{R}^{n \times p}$ is the standard input matrix. Any tabular dataset with n samples and p features becomes an $n \times p$ matrix fed into models.

Common Matrix Types

Row Matrix ($1 \times n$)

A single row — equivalent to a row vector. Used to represent a single data sample or a weight vector transposed.

Column Matrix ($m \times 1$)

A single column — equivalent to a column vector. The standard representation of a data point or parameter vector.

Zero Matrix $\mathbf{0}$

All entries are 0. The additive identity for matrices:
 $A + \mathbf{0} = A$.

Diagonal Matrix

Non-zero entries only on the main diagonal. Scales each axis independently — computationally efficient.

Rectangular Matrix ($m \times n, m \neq n$)

Most real datasets. The data matrix $X \in \mathbb{R}^{n \times p}$ is almost always rectangular since $n \neq p$.

Square Matrix ($n \times n$)

Required for determinants, eigenvalues, and matrix inverses. Covariance matrices are always square.

Scalar Matrix

A diagonal matrix with all diagonal entries equal to the same scalar c . Equivalent to scalar multiplication: cI .

Identity Matrix I

Diagonal entries all 1, off-diagonal all 0. The multiplicative identity:
 $AI = IA = A$ for any compatible A .

Matrix Addition and Scalar Multiplication

Matrix Addition

Requirement: Both matrices must have identical dimensions ($m \times n$). Addition is undefined for matrices of different sizes.

Rule: Add corresponding entries element-wise – entry (i, j) of the sum equals $a_{ij} + b_{ij}$.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

Scalar Multiplication

Multiply **every entry** of the matrix by the scalar. The dimensions are unchanged.

$$3 \cdot \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 9 & 12 \end{bmatrix}$$

ML Applications

Bias Addition in Neural Networks

After computing XW , a bias matrix (or broadcast bias vector) is **added** element-wise to the activation matrix: $Z = XW + B$.

Gradient Descent Weight Update

The learning rate η **scales** the gradient matrix, then the result is **subtracted** from the weight matrix: $W \leftarrow W - \eta \nabla_W \mathcal{L}$.

Batch Normalization

Subtracts the mean matrix and divides by the standard deviation matrix – both element-wise matrix operations.

- ❏ Matrix addition and scalar multiplication satisfy all vector space axioms – the set of all $m \times n$ matrices is itself a vector space.

Matrix Multiplication Rule

The Dimension Rule

$$(m \times n)(n \times p) = (m \times p)$$

The **inner dimensions must match**. The result has the outer dimensions. If the inner dimensions differ, multiplication is undefined.

Example: $(2 \times 3)(3 \times 4) = (2 \times 4) \checkmark$

Invalid: $(2 \times 3)(4 \times 5)$ – inner dimensions $3 \neq 4$ ✗

How Each Entry Is Computed

Entry (i, j) of the result $C = AB$ is the **dot product** of row i from A with column j of B :

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

Critical Properties

Not Commutative

In general, $AB \neq BA$. Order matters – reversing the order of multiplication typically produces a completely different matrix (or may be undefined).

Associative

$(AB)C = A(BC)$ – grouping does not change the result. This allows efficient computation ordering.

Why It Matters in ML

Every **forward pass** through a neural network layer is a matrix multiplication. The dimension rule determines whether layer sizes are compatible – a mismatch causes an immediate error. Designing network architectures requires tracking dimensions through every layer.

- ❑ Matrix multiplication is the single most computationally intensive operation in deep learning – it is why GPUs and TPUs exist.

Worked Matrix Multiplication

Let $A = \begin{bmatrix} 2 & -1 & 3 \\ 4 & 0 & -2 \end{bmatrix}$ (2×3) and $B = \begin{bmatrix} 1 & 5 \\ 2 & -3 \\ 4 & 1 \end{bmatrix}$ (3×2). Since the inner dimensions match (both 3), AB is 2×2 .

Step-by-Step Computation

1

Row 1 $\times$ Col 1

$$2(1) + (-1)(2) + 3(4) = 2 - 2 + 12 = \mathbf{12}$$

2

Row 1 $\times$ Col 2

$$2(5) + (-1)(-3) + 3(1) = 10 + 3 + 3 = \mathbf{16}$$

3

Row 2 $\times$ Col 1

$$4(1) + 0(2) + (-2)(4) = 4 + 0 - 8 = \mathbf{-4}$$

4

Row 2 $\times$ Col 2

$$4(5) + 0(-3) + (-2)(1) = 20 + 0 - 2 = \mathbf{18}$$

Result

$$AB = \begin{bmatrix} 12 & 16 \\ -4 & 18 \end{bmatrix}$$

Each entry of AB is a dot product of one row of A with one column of B . The result is a 2×2 matrix as predicted by the dimension rule.

Key Takeaway

To compute entry (i, j) : take row i of the left matrix and column j of the right matrix, multiply corresponding elements, and sum. Repeat for every (i, j) pair.

Important Matrix Properties

Non-Commutativity

$AB \neq BA$ in general. Concrete example:

$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 2 \\ 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix}$$

The two products are different — order matters.

Associativity

$(AB)C = A(BC)$ — grouping does not change the result. This allows us to choose the most computationally efficient order of operations.

Transpose

A^T swaps rows and columns of A :

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \Rightarrow A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

A 2×3 matrix becomes 3×2 after transposition.

Transpose of a Product

$$(AB)^T = B^T A^T$$

The order **reverses** when transposing a product. This is essential in deriving gradient formulas during backpropagation.

Symmetric Matrix

A matrix is **symmetric** if $A = A^T$. Covariance matrices and kernel matrices in ML are always symmetric — a property exploited by PCA and kernel methods for efficient computation.

Matrices as Transformations

A matrix A applied to a vector $\mathbf{x}$ produces a new vector $A\mathbf{x}$ — a **linear transformation** of the input space.

Worked Example: Scaling Transformation

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \Rightarrow A\mathbf{x} = \begin{bmatrix} 4 \\ 9 \end{bmatrix}$$

This diagonal matrix stretches the x-axis by a factor of 2 and the y-axis by a factor of 3. The input point (2, 3) is mapped to the output point (4, 9).

Types of Linear Transformations

Rotation

Rotates vectors by a fixed angle around the origin.

Reflection

Flips vectors across a line or plane through the origin.

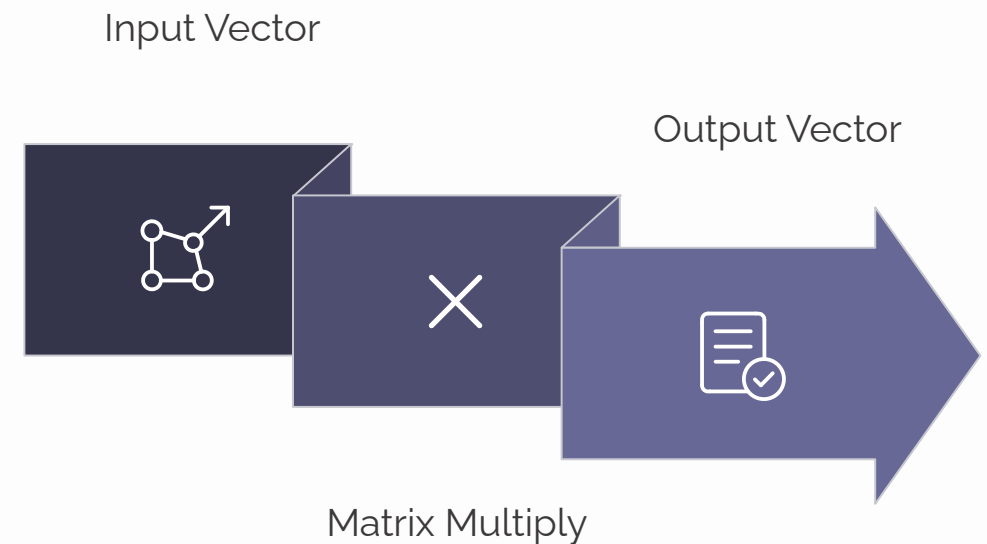
Shear

Slants vectors — one component shifts proportionally to another.

Projection

Collapses vectors onto a lower-dimensional subspace.

The Transformation Pipeline



ML Relevance

Each layer of a neural network applies a **learned linear transformation** (matrix multiplication) followed by a **non-linear activation function**. The network learns which sequence of transformations makes the data linearly separable in the final layer.

The power of deep learning comes from composing many such transformations — each matrix reshapes the data representation until the task becomes tractable.

Matrix Multiplication in Machine Learning

Layer Computation: XW

$$X \in \mathbb{R}^{100 \times 10}, \quad W \in \mathbb{R}^{10 \times 5} \quad \Rightarrow \quad XW \in \mathbb{R}^{100 \times 5}$$

100 Samples

Each row of X is one data point – 100 samples processed simultaneously.

10 Input Features

Each column of X is one feature – 10 dimensions in the input space.

5 Output Features

Each column of W defines one output dimension – 5 dimensions in the output space.

Transformation

Matrix multiplication projects all 100 samples from a 10-dimensional input space into a 5-dimensional output space in a single operation.

Batch Processing & Hardware Acceleration

All 100 samples are transformed **simultaneously** in a single matrix multiplication. This is fundamentally different from processing one sample at a time in a loop.

Why GPUs Dominate ML Training

GPUs contain thousands of cores optimized for exactly this operation – massively parallel matrix multiplication. A modern GPU can perform trillions of floating-point operations per second (TFLOPS) on matrix computations.

Training a large neural network without GPU acceleration would take months instead of hours. The entire hardware ecosystem of ML – GPUs, TPUs, and custom AI chips – exists to accelerate matrix multiplication.

- ❏ The dimension rule is not just a mathematical constraint – it is an engineering specification that determines hardware compatibility and memory layout.

Neural Networks and Images

The Layer Equation

$$Z = XW + b, \quad A = f(Z)$$

X — Input Data Matrix
($n \times d$)

n samples, each with d features. One row per sample.

W — Learned Weight Matrix ($d \times k$)

One column per output neuron. Updated by gradient descent during training.

b — Bias Vector

Broadcast across all n samples. Shifts the activation independently of the input.

f — Activation Function

Applied element-wise to Z .
Examples: ReLU ($\max(0, z)$), sigmoid, tanh.

Images as Matrices

Grayscale image: Each pixel is a single intensity value (0–255):

$$\begin{bmatrix} 0 & 50 & 255 \\ 20 & 200 & 100 \\ 255 & 150 & 30 \end{bmatrix}$$

A 3×3 grayscale image — in practice, images are hundreds or thousands of pixels wide and tall.

RGB Images: 3D Tensors

A color image is represented as a 3D tensor $H \times W \times 3$ — one $H \times W$ matrix per color channel (Red, Green, Blue).

Convolutional Neural Networks

CNNs apply learned **filter matrices** to image matrices via convolution — a structured form of matrix multiplication — to detect edges, textures, shapes, and objects at multiple scales.

Recommendation Systems

The User-Item Matrix

$$R = \begin{bmatrix} 5 & 4 & ? \\ 3 & ? & 5 \\ ? & 2 & 4 \end{bmatrix}$$

Rows are **users**, columns are **movies** (or items). Each entry is a rating.

User / Movie	M1	M2	M3
User 1	5	4	?
User 2	3	?	5
User 3	?	2	4

Known entries: Ratings already given. **Missing entries (?)**: Ratings to be predicted — the core recommendation problem.

Matrix Factorization

$$R \approx UV^T$$

Approximate the rating matrix as the product of two low-rank matrices: $U \in \mathbb{R}^{n \times k}$ encodes **user preferences** in a latent space, and $V \in \mathbb{R}^{m \times k}$ encodes **item attributes**.

Why Matrices?

Scale

Netflix has ~250M users and ~15K titles — the rating matrix has billions of potential entries. Linear algebra provides the only tractable tools at this scale.

Sparsity

Most entries are missing (users rate only a tiny fraction of items). Matrix factorization fills in the gaps by learning latent structure.

Latent Factors

The k latent dimensions capture hidden patterns — genre preferences, quality sensitivity — without being explicitly programmed.

Integration & Review

Bringing vectors, vector spaces, and matrices together in a complete ML example

1

Data

Raw observations stored as feature vectors in $\mathbb{R}^p$

2

Vectors

Each sample is a vector; operations define similarity and distance

3

Matrices

Samples stacked into X ; weights stored in W

4

Transformations

Matrix multiplication maps inputs to predictions

5

Learning

Gradient descent updates W to minimize loss

Complete ML Example

Putting it all together: a basic linear model using matrix–vector multiplication – the core computation of linear regression, logistic regression, and the first layer of any neural network.

Setup

Data matrix (3 samples, 2 features each):

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \in \mathbb{R}^{3 \times 2}$$

Weight vector (one weight per feature):

$$W = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \in \mathbb{R}^2$$

Prediction

$$XW = \begin{bmatrix} 1(2) + 2(3) \\ 3(2) + 4(3) \\ 5(2) + 6(3) \end{bmatrix} = \begin{bmatrix} 2 + 6 \\ 6 + 12 \\ 10 + 18 \end{bmatrix} = \begin{bmatrix} 8 \\ 18 \\ 28 \end{bmatrix}$$

Interpretation

Each Output Value

The model's prediction for one sample – a weighted sum of its features. Sample 1 predicts 8, Sample 2 predicts 18, Sample 3 predicts 28.

Weighted Sum

Each prediction is a dot product of the sample's feature vector with the weight vector – exactly $\hat{y}_i = \mathbf{w}^T \mathbf{x}_i$.

Batch Efficiency

All three predictions are computed in a single matrix–vector multiplication – no loop required. Scale to millions of samples with the same operation.

Concepts Unified

Vectors: $\mathbf{x}_i$ and $\mathbf{w}$ are vectors in $\mathbb{R}^2$. **Vector space:** $\mathbb{R}^2$ guarantees all operations are valid. **Matrix:** X stacks all samples; multiplication computes all predictions at once.

Final Takeaways

The Three Pillars

Vectors

Represent data, features, directions, and learned parameters. Every data point, every weight, every gradient is a vector. Operations — addition, scaling, dot product, norm — define how we measure, compare, and combine information.

Vector Spaces

Provide the mathematical setting that guarantees vector operations remain valid and meaningful. Feature spaces, weight spaces, and embedding spaces are all vector spaces — this structure is what makes gradient descent and linear algebra tools work correctly.

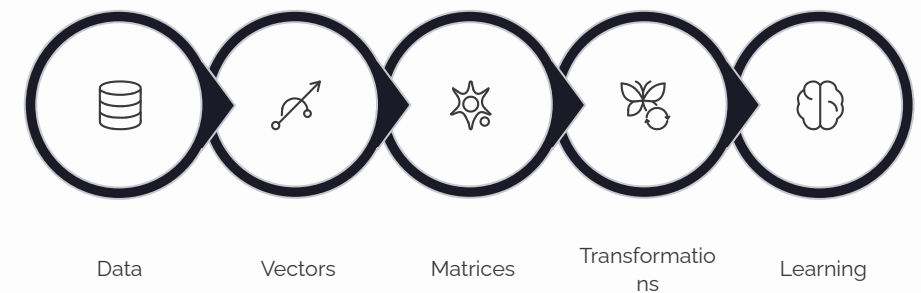
Matrices

Store datasets, encode linear transformations, and carry learned weights. Every forward pass, every parameter update, every dataset is a matrix operation. The dimension rule governs all of ML architecture design.

AI/ML Uses All Three

Features, weights, predictions, images, embeddings, and learning algorithms all rely on vectors and matrices — there is no ML without linear algebra.

Concept Map



Coming Next

Determinants & Inverses

When does a matrix have an inverse? What does the determinant measure?

Eigenvalues & Eigenvectors

The directions a matrix preserves — the foundation of PCA and spectral methods.

SVD

Singular Value Decomposition — the deepest factorization, powering PCA, compression, and recommendation systems.